

Table

👤 소유자	👤 Hyejun Park
🕒 생성 일시	@2026년 1월 7일 오후 11:04
☑️ 시험언급	☐

- CPU : Apple M3
- OS : macOS 14.5

Compiler	Cores/Th.	Scale	S-Thread	threads	MPI	Note
1.90.0	8/8	0	3.46	25.42	3.48	512×512
1.90.0	8/8	1	3.42	22.79	3.46	1024×1024
1.90.0	8/8	2	3.45	25.68	3.38	2048×2048
1.90.0	8/8	3	3.46	24.84	3.29	4096×4096
1.90.0	8/8	4	-	-	-	timeout
1.90.0	8/8	5	-	-	-	timeout

Script

```
#!/bin/bash
# simple_find_max.sh - macOS version
#
# Purpose:
# This script benchmarks a Mandelbrot renderer binary by increasing
# the problem size (controlled by parameter T) until it fails or times out.
# It performs:
# - A quick sanity test
# - System information logging
# - Performance benchmarking with timeout handling
# - Automatic parsing of performance results
#
# Target platform:
# - macOS (uses sysctl instead of /proc)
# - Assumes Rust + Cargo environment

echo "=== Mandelbrot Max Scale Finder ==="
echo ""

# -----
# Print current working directory
# -----
echo "Current directory: $(pwd)"
echo ""

# -----
# Check if the compiled binary exists
# If not found, build it in release mode
# -----
BINARY="./target/release/ex10-522142-park-hyejun"
if [ ! -f "$BINARY" ]; then
```

```

    echo "Binary not found at $BINARY"
    echo "Building..."
    cargo build --release
fi

echo "Binary: $BINARY"
echo ""

# -----
# Quick sanity check with a very small problem size
# This ensures the binary runs before heavy tests
# -----
echo "=== Quick Test (T=2) ==="
OUTPUT_FILE="quick_test.pgm"
echo "Running: $BINARY $OUTPUT_FILE 2"
"$BINARY" "$OUTPUT_FILE" 2

if [ $? -eq 0 ]; then
    echo "Quick test: SUCCESS"
    rm -f "$OUTPUT_FILE"
else
    echo "Quick test: FAILED"
fi
echo ""

# -----
# System information (useful for benchmark reports)
# -----
echo "=== System Information ==="
echo "CPU: $(sysctl -n machdep.cpu.brand_string)"
echo "OS: macOS $(sw_vers -productVersion)"
echo "Cores: $(sysctl -n hw.physicalcpu)/$(sysctl -n hw.logicalcpu)"
MEM_BYTES=$(sysctl -n hw.memsize)
MEM_GB=$((MEM_BYTES / 1024 / 1024 / 1024))
echo "Memory: ${MEM_GB}GB"
echo "Rust: $(rustc --version | cut -d' ' -f2)"
echo ""

# -----
# Performance benchmarking loop
# T controls the image size: SIZE = 2^(T + 9)
# Each run is limited to 30 seconds to avoid hangs
# -----
echo "=== Performance Tests ==="
echo "T | Size | Seq Mpx/s | Thread Mpx/s | MPI Mpx/s"
echo "---|-----|-----|-----|-----"

MAX_T=0

for T in 0 1 2 3 4 5; do
    # Compute image resolution for current T
    SIZE=$((2 ** (T + 9)))
    OUTPUT_FILE="bench_T${T}.pgm"

```

```

echo -n "Testing T=$T (${SIZE}x${SIZE})..."

# -----
# Run the program in the background and capture output
# Output is redirected to a file for later parsing
# -----
"$BINARY" "$OUTPUT_FILE" "$T" > "output_T${T}.txt" 2>&1 &
PID=$!

# -----
# Timeout handling:
# Wait 30 seconds, then kill the process if still running
# (macOS does not have GNU timeout by default)
# -----
sleep 30

if kill -0 $PID 2>/dev/null; then
    # Process still running → timeout
    kill $PID 2>/dev/null
    wait $PID 2>/dev/null
    echo " × (timeout)"
else
    # Process finished within time
    wait $PID
    EXIT_CODE=$?

    if [ $EXIT_CODE -eq 0 ]; then
        echo " ✓"

        # -----
        # Parse performance numbers from program output
        # Expected format:
        # Algorithm X ... = <value> Mpx/s
        # -----
        SEQ_MPS=$(grep "Algorithm 1" "output_T${T}.txt" | tail -1 | sed -E 's/.*= ([0-9]+\.[0-9]+) Mpx\s.*/\1/')
        THREAD_MPS=$(grep "Algorithm 2" "output_T${T}.txt" | tail -1 | sed -E 's/.*= ([0-9]+\.[0-9]+) Mpx\s.*/\1/')
        MPI_MPS=$(grep "Algorithm 3" "output_T${T}.txt" | tail -1 | sed -E 's/.*= ([0-9]+\.[0-9]+) Mpx\s.*/\1/')

        if [ -n "$SEQ_MPS" ]; then
            printf "%1d | %5dx%-5d | %9.2f | %12.2f | %9.2f\n" \
                $T $SIZE $SIZE ${SEQ_MPS:-0} ${THREAD_MPS:-0} ${MPI_MPS:-0}
            MAX_T=$T
        else
            echo " (no performance data found)"
        fi
    else
        echo " × (exit code: $EXIT_CODE)"
    fi
fi

# -----

```

```

# Cleanup generated files before next iteration
# -----
rm -f "$OUTPUT_FILE"
sleep 2
done

echo ""
echo "=== Summary ==="
echo "Maximum successful T: $MAX_T"

if [ $MAX_T -gt 0 ]; then
    MAX_SIZE=$((2 ** (MAX_T + 9)))
    MAX_PIXELS=$((MAX_SIZE * MAX_SIZE))
    MAX_MB=$((MAX_PIXELS / 1024 / 1024))

    echo "Maximum size: ${MAX_SIZE}x${MAX_SIZE}"
    echo "Maximum pixels: $((MAX_PIXELS / 1000000)) Mpx"
    echo "Estimated memory usage: ${MAX_MB} MB"
fi

# -----
# Final cleanup of temporary benchmark artifacts
# -----
rm -f output_T*.txt bench_*.pgm

```

Result

```

=== System Information ===
CPU: Apple M3
OS: macOS 26.1
Cores: 8/8
Memory: 16GB
Rust: 1.90.0

=== Performance Tests ===
T | Size | Seq Mpx/s | Thread Mpx/s | MPI Mpx/s
--|-----|-----|-----|-----
Testing T=0 (512×512)... ✓
0 | 512×512 | 3.46 | 25.42 | 3.48
Testing T=1 (1024×1024)... ✓
1 | 1024×1024 | 3.42 | 22.79 | 3.46
Testing T=2 (2048×2048)... ✓
2 | 2048×2048 | 3.45 | 25.68 | 3.38
Testing T=3 (4096×4096)... ✓
3 | 4096×4096 | 3.46 | 24.84 | 3.29
Testing T=4 (8192×8192)... × (timeout)
Testing T=5 (16384×16384)... × (timeout)

=== Summary ===
Maximum successful T: 3

```